

Homework 6

Problem 6. Consider the non-linear equation for x :

$$\int_0^x \frac{1}{\sqrt{2\pi}} e^{-t^2/2} dt = 0.45.$$

(a) Define

$$f(x) := \int_0^x \frac{1}{\sqrt{2\pi}} e^{-t^2/2} dt - 0.45.$$

Using Fundamental Theorem of Calculus, write down Newton's Method applied to the equation $f(x) = 0$.

- (b) Each step of Newton's Method derived in (a) requires an evaluation of $f(x)$, while we cannot evaluate $f(x)$. Instead, we apply Composite Trapezoidal Rule to estimate $f(x)$ with N subintervals. Write down the expression for the Composite Trapezoidal Rule with N subintervals to approximate $\int_0^x \frac{1}{\sqrt{2\pi}} e^{-t^2/2} dt$ (denoted by $I_{\text{trap}}[g; x; N]$, where $g(t) = \frac{1}{\sqrt{2\pi}} e^{-t^2/2}$). Then, write down the modified version of Newton's Method where the integration is approximated by Composite Trapezoidal Rule.
- (c) Write a code to implement the method you derived above with $N = 50$ subintervals and initialization $x_0 = 0.5$. Stop the iteration when the approximation for the residual $|f(x_n)|$ is smaller than $\tau = 10^{-5}$ or a maximum number of iteration 100 is reached. Report the number of iterations and the approximated root (with 5 significant digits)

Solution to (a): Using FTC we have,

$$f'(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}.$$

So Newtons method would be:

$$x_{n+1} = x_n - \sqrt{2\pi} e^{x_n^2/2} \left[\int_0^{x_n} \frac{1}{\sqrt{2\pi}} e^{-t^2/2} dt - 0.45 \right].$$

□

Solution to (b): The Composite Trapezoidal Rule is given by:

$$\int_0^x g(t) dt \approx \frac{h}{2} \left(g(0) + 2 \sum_{i=1}^{N-1} g(x_i) + g(x) \right).$$

Note that $h = x/N$, and $x_i = ih = (ix)/N$. So we have,

$$I_{\text{trap}}[g, x, N] = \frac{x}{2N} \left(\frac{1}{\sqrt{2\pi}} + 2 \sum_{i=1}^{N-1} g\left(\frac{ix}{N}\right) + g(x) \right).$$

Plugging this into the Newton's method formula we get,

$$x_{n+1} = x_n - \sqrt{2\pi} e^{x_n^2/2} \left(\frac{x_n}{2N} \left[\frac{1}{\sqrt{2\pi}} + 2 \sum_{i=1}^{N-1} g\left(\frac{ix_n}{N}\right) + g(x) \right] - 0.45 \right).$$

□

Solution to (c): We have the following code:

```
import numpy as np

#constants
N = 50
e = 10**(-5)
div = 10**5

#integrand
def g(x):
    return 1/(np.sqrt(2*np.pi)) * np.exp(-(x**2)/2)

#ftc for readability
def df(x):
    return g(x)

#composite trapezoidal rule
def I(x):
    sum = 0
    for i in range(1, N):
        sum += g(i*x/N)
    return (x/(2*N))*(g(0) + 2*sum + g(x))

#apx of integral subtracted by 0.45
def f(x):
    return I(x) - 0.45

def newton (p0):
    pn = p0
    it = 0
    #check if we are done
    while (np.abs(f(pn)) > e and it < 100):
        if (np.abs(f(pn)) > div):
            return ("diverged", it)

    #generate sequence
    pn = pn - f(pn)/df(pn)
```

```
        it += 1
    return (pn, it)
```

```
#results
res = newton(0.5)
print("===== Results =====")

print(f"Root: {res[0]:.5g}, Iterations: {res[1]}")
```

Results: The root we find is approximately 1.6450 and we reached this root within 4 iterations.

□